
LogicWard

Live OT Drift-Detection & Incident-Response Platform

A unified, multi-site fabric that detects unauthorized PLC logic and process drift *before* it becomes operational or safety risk.

Two monitored plants, one detection fabric:

Site A — Thermal Power Plant (Raspberry Pi PLC, Rockwell L5X ladder, animated SCADA mimic)

Site B — GRFICS Chemical Reactor (OpenPLC Structured Text, live Unity 3D digital twin)

Every attack traverses the full lifecycle: *Execution* → *Register change* → *3D/gauge update* → *Drift detection* → *Alert* → *Evidence* → *MITRE ATT&CK for ICS* → *Timeline* → *Response* → *Rollback*.

Contents

1 Executive Summary 2

1.1 Headline capabilities 2

2 Problem Statement & Threat Model 2

2.1 Why OT drift is hard 2

2.2 Adversary capabilities modelled 2

2.3 Detection philosophy 3

3 System Architecture 3

3.1 The event bus is the spine 3

3.2 The site abstraction (process-agnostic) 3

4 Multi-Site Model 4

5 Attack Lifecycle & Data Flow 4

6 Detection Engine 4

6.1 Structural (program) detection 4

6.2 Register (process) detection 4

6.3 Signed baseline & tamper detection 5

7 Attack Catalogue 5

7.1 Site A — Thermal (7) 5

7.2 Site B — Chemical (5, demo-ordered, blast last) 5

8 Data Model (Entity–Relationship) 6

9 Role-Based Access Control 6

10 MITRE ATT&CK for ICS Mapping 6

11 SOC Dashboard & Analytics 7

12 Deployment Topology 7

13 Verification & Results 7

14 Roadmap 8

1 Executive Summary

Operational-Technology (OT) attacks do not announce themselves. A single unauthorized change to a Programmable Logic Controller (PLC) — a lowered trip setpoint, an inverted safety comparator, a hijacked output coil — can sit invisibly in a running plant until it causes a trip, a spill, or a catastrophe. **LogicWard** is a live drift-detection appliance that watches the running control logic and the live process state, and raises *severity-ranked*, *MITRE-mapped*, *evidence-backed* alerts the instant either drifts from a cryptographically signed, approved baseline.

The platform is deliberately **multi-site and multi-vendor**. It simultaneously monitors a *thermal power plant* on real Raspberry Pi hardware (Rockwell L5X ladder logic) and a *chemical reactor* rendered as a live Unity 3D digital twin (OpenPLC Structured Text), proving that the detection fabric is *process-agnostic*: the same event bus, severity model, MITRE mapping and evidence log serve two entirely different plants under one SOC console.

The prime architectural invariant. The PLC “program” is **data to baseline, diff and mutate** — **it is never executed**. LogicWard does not interpret ladder logic; the live Modbus registers provide the physical reality underneath. This is what keeps the appliance safe, fast and false-positive-free.

1.1 Headline capabilities

- **Structural + register drift detection** against an HMAC-SHA256 signed baseline, naming six PLC mutations plus raw register/coil tampering.
- **Unified event fabric** — one Event contract, three detection planes (cyber / physical / resource), enriched on emit with severity, identity and MITRE.
- **Two live plants** under one dashboard with a site selector, a shared alert timeline, and per-site forensic PDF export.
- **Twelve real attacks** (7 thermal, 5 chemical) fired from a red-team console over genuine unauthenticated Modbus / program-download channels.
- **Production-grade SOC UI** — risk score, attack timeline, system health, rollback status, live trend sparklines.
- **Capability-based RBAC** across six OT/ICS roles, enforced server- and client-side.
- **154 automated integration checks** across nine suites.

2 Problem Statement & Threat Model

2.1 Why OT drift is hard

IT security assumes patchable, observable, restartable assets. OT inverts every assumption: PLCs run unauthenticated legacy protocols (Modbus has no authentication), cannot be patched mid-process, and a “reboot” may mean a plant trip. The adversary’s goal is rarely data theft; it is *physical consequence* — defeat a safety interlock, then let physics do the damage.

2.2 Adversary capabilities modelled

- **Unauthenticated Modbus writes** (FC05/FC06/FC10) — move setpoints, force coils.
- **Engineering-workstation compromise** — download a mutated PLC program.
- **Network foothold** — rogue master / device on the OT segment; denial of service.

- **Physical access** — cabinet tamper, cable pull (physical plane).

2.3 Detection philosophy

LogicWard assumes the attacker is *already inside* and can write to the PLC. It does not try to keep them out; it makes any deviation from the approved state *immediately visible, attributable and reversible*. Detection is a structural *diff*, not anomaly-guessing — which is why a harmless program re-export (new timestamps, same logic) hashes identically and raises nothing, while a real logic change is caught on the first scan.

3 System Architecture

LogicWard is a Python package (`logicward/`) built on Flask and a raw-socket Modbus stack. Figure 1 shows the end-to-end fabric: red-team attack channels on the left, the two monitored sites in the middle, and the shared detection/evidence/presentation plane on the right.

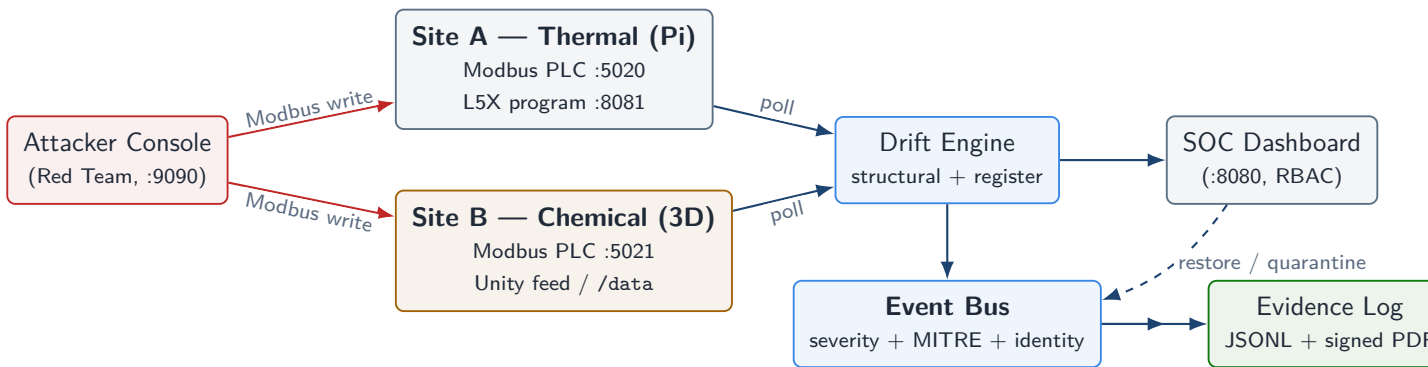


Figure 1: End-to-end architecture. Attacks are real unauthenticated writes; both plants are polled by one transport-agnostic drift engine that feeds a single enriched event bus, which fans out to the evidence log and the role-gated SOC dashboard.

3.1 The event bus is the spine

Every detector — cyber, physical, resource, and the response engine — emits the *same* Event object. On `emit()` the bus validates, is idempotent by `event_id`, enriches (computes severity, attaches MITRE, identity, sequence), and fans out to three sinks: the append-only JSONL evidence log, the dashboard poll buffer, and live subscribers.

```

Event = {
    event_id, type,                # e.g. "cyber.setpoint_drift" | "physical.rogue_device"
    timestamp, source, severity,
    details { ... , site },        # site attributes the event to a plant
    identity { who, mac, channel },
    mitre { tactic, technique_id, technique_name, verified },
    seq, received_at
}
  
```

3.2 The site abstraction (process-agnostic)

A single seam — `engine/sources.py` — makes the engine transport- and process-agnostic: it only needs `program_source() -> bytes` and `register_source() -> {holding, coils}`. `EmbeddedPlant` runs an in-process PLC (one-machine dev/demo); `RemotePlant` reads a real Pi over Modbus + HTTP. Site B (`sites/grfics/`) plugs into the same seam as a second site.

4 Multi-Site Model

	Site A — Thermal Power Plant	Site B — GRFICS Chemical Reactor
Hardware	Raspberry Pi (real)	Laptop (simulated)
Vendor / program	Rockwell L5X ladder	OpenPLC Structured Text
Detection surface	structural (program) + register	register plane
Visualization	animated SVG SCADA mimic	live Unity 3D digital twin
Modbus	TCP :5020 (unauthenticated)	TCP :5021 (unauthenticated)
Site id	thermal-pi	grfics-chem

Table 1: The two monitored sites. Both feed one bus; events are attributed via `details.site`.

Why this matters for the pitch. The claim is not “I detect drift on my simulator.” It is “I detect drift across *vendors, programming languages and physical processes* on one fabric.” Adding a water plant or a substation is a new *site profile*, not a rewrite.

5 Attack Lifecycle & Data Flow

Every attack — on either site — traverses the same ten-stage lifecycle (Figure 2). This is the backbone of the demonstration: an observer can point to each stage on screen.

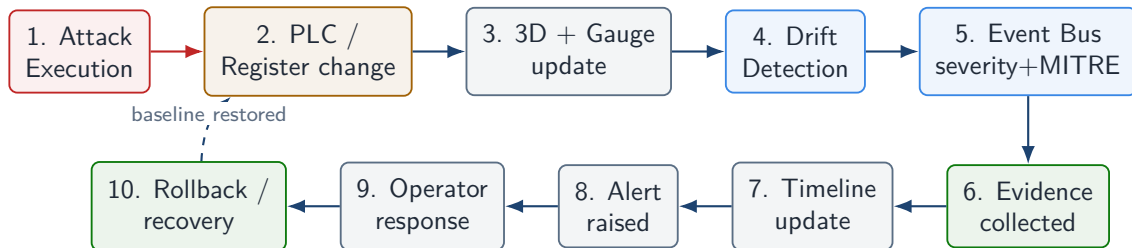


Figure 2: The ten-stage attack lifecycle every scenario follows. Stages 4–7 are the automated LogicWard core; stages 9–10 are the role-gated response (acknowledge, quarantine, safe-state, restore baseline).

6 Detection Engine

Detection runs in a one-second loop with two surfaces.

6.1 Structural (program) detection

The live L5X is parsed and *canonicalized* (`engine/l5x.py`): volatile attributes (export/edit dates, revisions, CRC, comments) are stripped, neutral-text ladder such as `XIC(..)[LES(..)]OTE(..)`; is parsed into instruction/operand structs, and a `structural_hash` is produced. A rung-by-rung diff against the signed baseline names six mutations:

6.2 Register (process) detection

Live Modbus holding registers and control coils are diffed against the baseline snapshot. Input registers (sensor values) are *intentionally not* diffed — they carry live process noise; only operator-controllable holdings and coils are integrity-checked, which is why the plant sits silent at rest and only speaks when tampered.

Mutation	Meaning	Severity	MITRE ICS
condition_stripping	safety input removed from a rung	critical	T0889
logic_inversion	comparator/contact flipped (LES $\leftrightarrow$ GRT)	critical	T0889
coil_hijack	output coil repointed to another actuator	critical	T0889
rung_injection	unauthorized new rung added	high	T0843
setpoint_drift	threshold / setpoint tag value changed	high	T0836
register_change	raw holding-register / coil write over Modbus	low-med	T0855

Table 2: The six named mutations, their severity band and verified MITRE ATT&CK for ICS technique.

6.3 Signed baseline & tamper detection

The approved configuration (program + structural hash + setpoints + register snapshot) is captured into a manifest and signed with **HMAC-SHA256** (`engine/baseline.py`). Editing the locked manifest breaks the signature; a `watchdog` file-integrity monitor watches both the live program file and the signed baseline and re-verifies on change. This is integrity, not access control — and it is honestly framed as such.

7 Attack Catalogue

Twelve real attacks fire over genuine channels. Thermal program mutations use the unauthenticated program-download endpoint; register/coil attacks use raw Modbus.

7.1 Site A — Thermal (7)

Attack	Channel	Effect	MITRE
Setpoint Drift	Modbus FC06	Drum-level trip setpoint 220 $\rightarrow$ 40 mm (blinds protection)	T0836
Logic Inversion	Program	drum-level comparator LES $\rightarrow$ GRT (trips backwards)	T0889
Condition Stripping	Program	remove <code>Plant_Running</code> interlock from fuel trip	T0889
Coil Hijack	Program	repoint <code>Feedwater_Trip</code> $\rightarrow$ <code>Cooling_Pump_Stop</code>	T0889
Rung Injection	Program	inject backdoor rung unlatching turbine overspeed trip	T0843
Force Coil	Modbus FC05	force <code>Fuel_Valve_Open</code> off (flame-loss trip)	T0855
DDoS Flood	Modbus FC03	multi-threaded read flood, CPU spike	T0814

7.2 Site B — Chemical (5, demo-ordered, blast last)

Each chemical attack drives a *distinct* live signature on the SOC gauges (and, where the compiled Unity binary supports it, on the 3D scene). They are ordered for a live pitch that escalates to a catastrophic finale.

#	Attack	Distinct signature	MITRE
1	Quality Sabotage	feed ratio skewed; composition/colour shifts, no alarm (stealth)	T0855
2	Feed Pump Trip	pump 1 off; reactor starved, liquid level <i>drains</i>	T0855
3	Tank Overfill	product shut, feeds high; level <i>rises</i> to overflow + trip	T0855
4	E-Stop Injection	ESD coil forced; plant halts, all valves safe-state	T0855
5	Pressure Redline (BLAST)	<i>all</i> trips defeated + feeds open + purge shut; pressure to the ceiling — the reactor blasts	T0836

Honest 3D limitation. The GRFICS Unity scene is a *pre-compiled WebGL binary*; its process animations (liquid level, colour) are driven only where the binary already maps the JSON keys. The pressure explosion animates; level/colour depend on the binary and are fully solved by a Unity recompile. *All* per-attack distinctiveness is guaranteed on the live SOC gauges, detection feed and timeline — where the telemetry is actually read during a pitch.

8 Data Model (Entity–Relationship)

LogicWard’s runtime state is a small, clean model (Figure 3). **Event** is the central entity; **Site** attributes it to a plant; **Baseline** is the signed reference the drift engine diffs against; **User**→**Role**→**Capability** governs access.

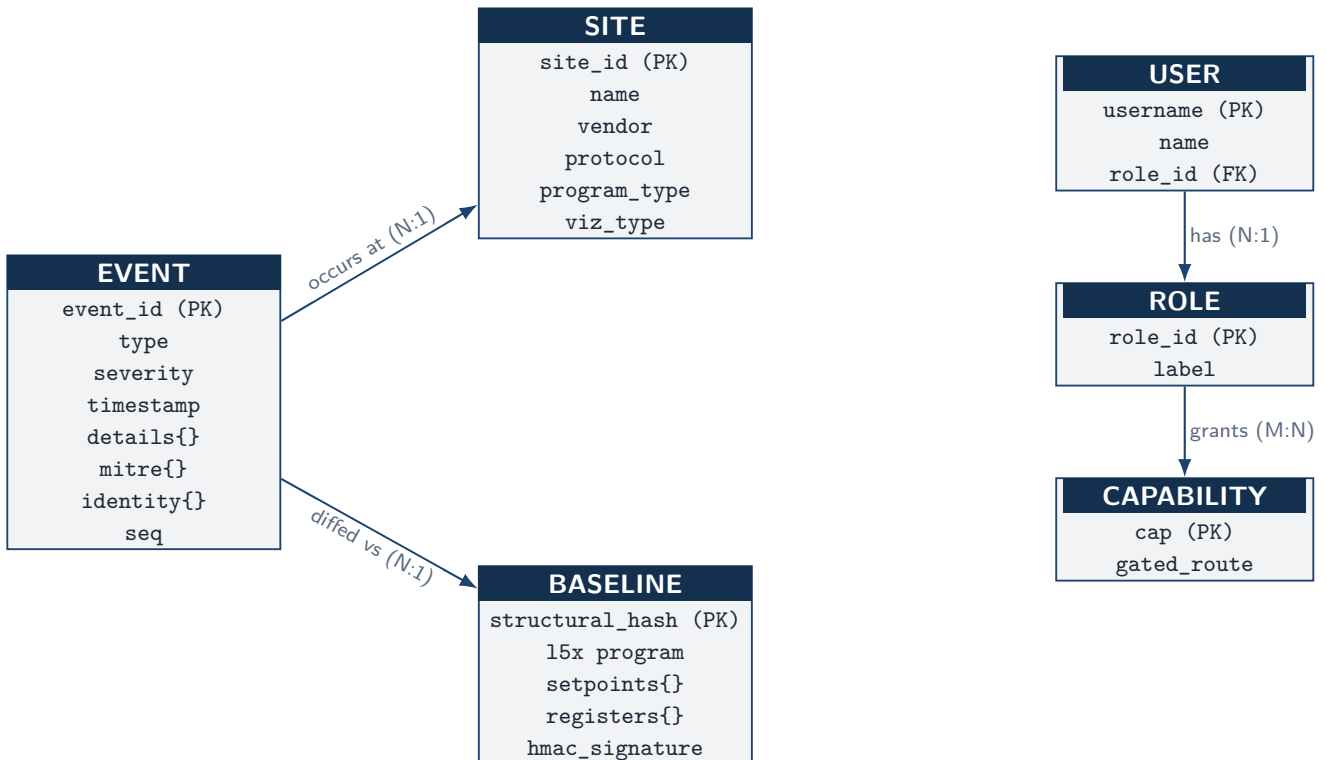


Figure 3: Runtime data model. Events are attributed to a Site and diffed against a signed Baseline; access is a User→Role→Capability chain (capability, not rank).

9 Role-Based Access Control

Access is **capability-based**, not a linear rank, because the six OT/ICS roles have different *scopes* — a Network Engineer may quarantine a rogue device but must not touch the PLC program; a Control Engineer is the reverse. Every action is gated server-side (HTTP 403) and client-side (the control is removed from the DOM).

10 MITRE ATT&CK for ICS Mapping

Mapping is a rule-based, explainable table (`engine/mitre_map.py`), verified against the live ICS matrix. Where no technique honestly fits (physical enclosure tamper, baseline-integrity tamper) it is marked N/A rather than assigned a fabricated ID.

Role (login)	Scope	ack	ack all	baseline	net. resp.	safe-state	evidence
Operator (operator)	control room	✓	✓				
Control Engineer (engineer)	PLC program	✓	✓	✓		✓	
Network Engineer (netsec)	infrastructure	✓	✓		✓		
SOC Analyst (soc)	oversight	✓	✓		✓	✓	✓
Vendor / OEM (vendor)	scoped, read-only						
CISO (ciso)	full authority	✓	✓	✓	✓	✓	✓

Table 3: Capability matrix for the six roles (verified end-to-end). Vendor is deliberately read-only and heavily monitored; CISO holds full cross-role authority.

Technique	ID	Tactic	LogicWard event types
Modify Parameter	T0836	Impair Process Control	<code>cyber.setpoint_drift</code>
Unauthorized Command Message	T0855	Impair Process Control	<code>cyber.register_change</code>
Modify Program	T0889	Persistence	inversion, stripping, coil hijack
Program Download	T0843	Lateral Movement	<code>cyber.rung_injection</code>
Rogue Master	T0848	Initial Access	<code>physical.rogue_device</code>
Denial of Service	T0814	Inhibit Response Function	DDoS, link down, CPU spike

Table 4: Verified ATT&CK for ICS techniques and the events that map to them.

11 SOC Dashboard & Analytics

The Flask dashboard (:8080) polls `/api/events?since=` and `/api/plant` (~1s; deliberately no WebSockets, for demo robustness) and presents a production-grade console:

- **Risk Score** — explainable 0–100 platform score (severity-weighted active alerts + baseline/program posture) with a LOW→CRITICAL band.
- **Attack Timeline** — severity-coloured temporal strip of events, site-filtered.
- **System Health** — per-site online status + event rollup (NOMINAL/WARNING/CRITICAL).
- **Baseline & Rollback status** — integrity, program drift count, restore availability.
- **Live trend sparklines** — rolling 60-sample history for the key thermal values.
- **Animated SCADA mimic** (thermal) — each event maps to a diagram node and redlines it with a pinned MITRE badge; click to acknowledge.
- **Live 3D digital twin** (chemical) — the Unity reactor beside live gauges.
- **Site selector** — Thermal / Chemical / All, driving one common alert timeline.

12 Deployment Topology

The demonstration runs on a laptop plus one Raspberry Pi over a LAN (Figure 4). The laptop hosts the SOC dashboard, the attacker console and the (local) chemical plant; the Pi runs the real thermal PLC, program endpoints and a sensor agent that POSTs physical/resource events back to the laptop’s ingest endpoint.

13 Verification & Results

LogicWard ships with **nine standalone integration suites** — **154 checks** — that spin up real servers on ephemeral ports and exercise real HTTP/Modbus paths (integration, not unit tests).

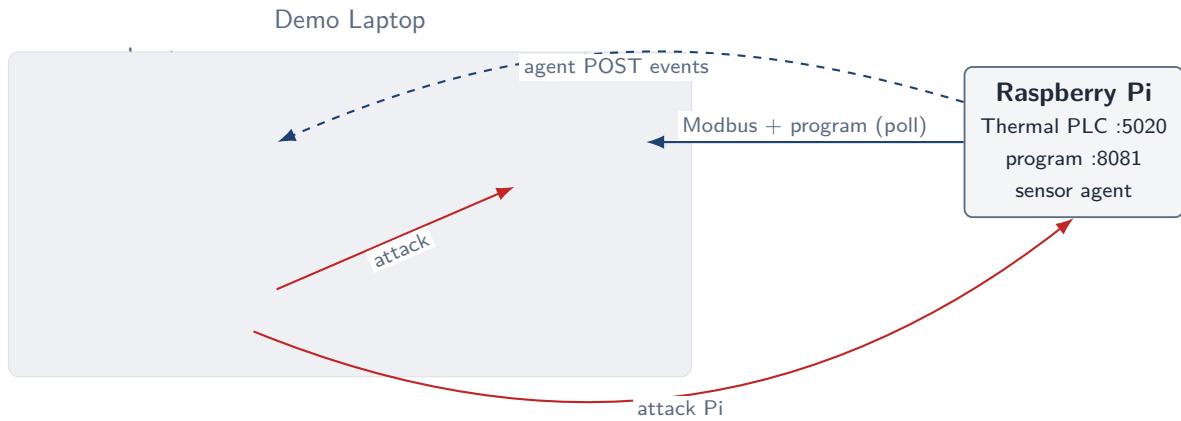


Figure 4: Two-machine deployment. Set `LOGICWARD_MULTISITE=0` on the Pi (thermal only); the laptop runs the unified dashboard, the red-team console and the chemical digital twin.

Suite	Checks	Covers
smoke_bus	15	event contract, severity, MITRE, evidence, idempotency
smoke_l5x	16	L5X parse, canonicalize, structural hash
smoke_plant	14	thermal Modbus server + setpoint-driven trips
smoke_drift	18	six mutations + register diff, no false positives
smoke_agent	10	physical/resource sensors + forwarder
smoke_dashboard	15	routes, RBAC (403 gating), response actions
smoke_attacker	10	attack framework + mutators
smoke_grfics	36	5 distinct chemical attacks, each own signature
smoke_multisite	20	unified feed, per-site evidence, attacker both sites
Total	154	

Table 5: The full verification matrix. Every attack’s detection, severity, MITRE and site-tagging is asserted; the chemical suite proves all five attacks produce distinct signatures.

14 Roadmap

- **Structured-Text program-drift** for the chemical PLC (a second `ProgramParser` behind the same canonical rung model — the six mutations apply unchanged).
- **Unity recompile** to hook liquid level / composition colour / e-stop strobes to the already-published JSON keys, unlocking full 3D per-attack animation.
- **New site profiles** — water treatment, electrical substation — as drop-in plugins.
- **ML-assisted risk scoring** layered above the explainable rule-based core.